Atividade Prática 1

CONTEXTO:
Nesta atividade prática, você testará requisições **HTTP (GET, POST, PUT e DELETE)** utilizando a **Poke Base API** localmente e o aplicativo **Bruno**.

Guia de instalação:
1. Baixar o Bruno: Acesse **https://www.usebruno.com/** e instale o app.
2. No terminal (na sua pasta de usuário) baixe o projeto da API com o comando:
git clone https://github.com/batistabjs/poke-base-api
3. No VS Code, abre a pasta **poke-base-api**
4. Ainda No VS Code, abra o terminal e execute os comandos:
npm install
npm start
A API rodará no endereço **http://localhost:8080**
5. No app Bruno, crie uma nova Collection chamada **Poke-Base-API**

DICA 1: Onde consultar os dados de pokémons já pré-cadastrados?
Se quiser conferir os Pokémons e usuários que já vêm salvos no projeto, abra os arquivos JSON na pasta **/dados** (**dados/pokemons.json** e **dados/usuarios.json**)

DICA 2: Dê ênfase à seção **"Exemplos de Uso"** no arquivo **README.md** do projeto; ela mostra exatamente como estruturar as URLs e os dados (JSON) das requisições.

DESAFIO:
Crie e salve as 10 requisições abaixo dentro da sua Collection no Bruno:

1. GET - Listar Pokémons
Método: GET | URL: http://localhost:8080/api/v1/pokemons
Objetivo: Listar todos os Pokémons (público).

2. GET - Buscar Pokémon por ID
Método: GET | URL: http://localhost:8080/api/v1/pokemons/1
Objetivo: Buscar os dados do Pokémon de ID 1.

3. GET - Filtrar por Tipo
Método: GET | URL: http://localhost:8080/api/v1/pokemons?tipo=eletrico
Objetivo: Filtrar a lista trazendo apenas Pokémons do tipo elétrico.

4. GET - Filtrar por Geração
Método: GET | URL: http://localhost:8080/api/v1/pokemons?geracao=1
Objetivo: Filtrar a lista trazendo apenas Pokémons da geração 1.

5. GET - Busca por Nome
Método: GET | URL: http://localhost:8080/api/v1/pokemons?busca=pikachu
Objetivo: Buscar Pokémons pelo termo "pikachu".

6. POST - Cadastrar Novo Usuário (Público)
Método: POST | URL: http://localhost:8080/api/v1/auth/cadastro
Body (JSON):

```
{
  "nome": "Seu Nome",
  "email": "aluno@pokebase.com",
  "senha": "123"
}
```

7. POST - Fazer Login e Obter Token (Público)
Método: POST | URL: http://localhost:8080/api/v1/auth/login
Body (JSON): Use o usuário seed admin@pokebase.com (senha 123456) ou o que criou no Desafio
Objetivo: Receber o token_gerado na resposta e copiá-lo.

8. POST - Criar Pokémon (Autenticado)
Método: POST | URL: http://localhost:8080/api/v1/pokemons
Tipo de Auth (no app Bruno): Selecione Bearer Token e cole o token obtido no Desafio 7.
Body (JSON): (Consulte os exemplos no README para os campos completos)

```
{
  "nome": "MeuPokemon",
  "numero": 999,
  "tipo_primario": "fogo",
  "geracao": 1,
  "regiao": "Kanto",
  "classe": "normal"
}
```

9. PUT - Atualizar Pokémon (Autenticado)
Método: PUT | URL: http://localhost:8080/api/v1/pokemons/1 (ou o ID do Pokémon criado)
Tipo de Auth (no app Bruno): Bearer Token.
Body (JSON):

```
{
  "descricao": "Nova descrição atualizada via requisição PUT"
}
```

10. DELETE - Deletar Pokémon (Autenticado)
Método: DELETE | URL: http://localhost:8080/api/v1/pokemons/1 (ou o ID do Pokémon criado) Tipo de Auth (no app Bruno): Bearer Token.
Objetivo: Remover o Pokémon indicado pelo ID.

ENTREGA:

O professor avaliará no seu computador o sucesso (ou não) de cada uma das atividades através de:

- Verificação da aplicação em execução no terminal (http://localhost:8080).
- Apresentação da Collection no Bruno com as 10 requisições devidamente salvas e testadas.
- Demonstração prática do envio das requisições usando o token nas rotas protegidas (POST, PUT e DELETE).